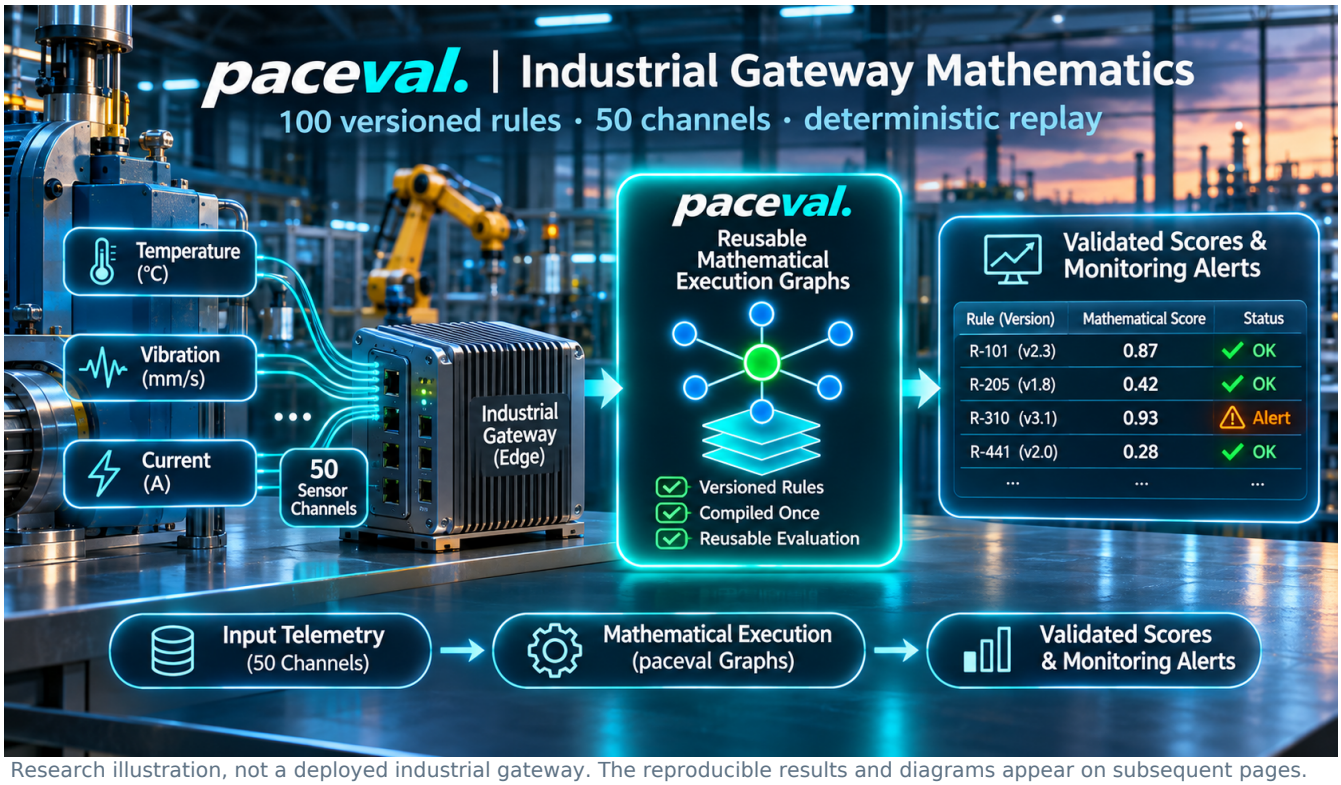
# Programmable Mathematical Rules on Industrial Gateways

Direct paceval C++ SDK integration, replayable telemetry, versioned rules and independently audited mathematical execution




Research illustration, not a deployed industrial gateway. The reproducible results and diagrams appear on subsequent pages.

Scientific research demonstrator | 11 November 2026

This experimental implementation converts a gateway-rule proposal into a functioning offline mathematical evaluation kernel. One hundred separately versioned computation graphs operate on fifty synthetic telemetry channels. The C++ host owns timestamps, quality screening, alert persistence, formula lifecycle and data export; paceval owns the prepared mathematical expressions and repeated numerical evaluation.

| Research scale | Verified outcome |
|---|---|
| 50 channels / 240 replay ticks | 12,000 synthetic input measurements |
| 100 prepared rules | 24,000 possible / 23,442 accepted rule evaluations |
| Independent audit | 600 90-digit point checks and 24 sampled TINC containments |

# 1. Research context and scientific boundary

Modern edge gateways acquire measurements, preserve time and quality semantics, transform data and forward events. The paceval opportunity is a narrowly scoped, reusable mathematical layer between validated inputs and operational decisions. An algebraic expression must not be confused with the protocol connector, control system, safety interlock or predictive-maintenance model.

The source research proposal specified 50 channels, 100 versioned rules and replay of gateway traffic. This package implements that research scale with openly documented synthetic signals rather than claiming access to industrial logs. The demonstration extends the scope with quality-state handling, a controlled rule update, host-owned alarm persistence and reproducible independent verification.

## Gateway integration boundary

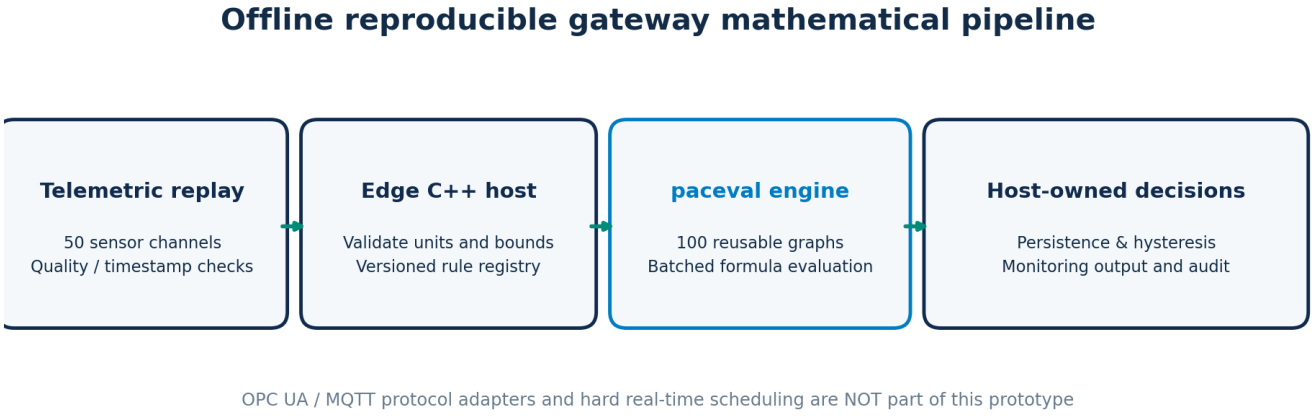



Figure 1. A clean separation between the input adapter, C++ validation, paceval execution graphs and host-owned monitoring logic.

OPC UA PubSub and MQTT/Sparkplug offer relevant industrial transport and state models; Eclipse Kura demonstrates how they can be hosted within gateways. None of these network stacks is embedded in this release. The tests exercise a CSV replay boundary, not a live protocol handshake or safety-critical installation.

# 2. Dataset and input contract

The fixed replay contains 240 ticks at a synthetic 100 ms cadence and 50 channel identities per tick. Each record preserves a strictly increasing timestamp, engineering units and one of the quality values GOOD, STALE, BAD, MISSING. No sensor calibration or hardware measurement is implied.

| Input field | Contract |
|---|---|
| Temperature T | Degrees Celsius, accepted range -30 to 180 |
| Vibration V | mm/s, accepted range 0 to 50 |
| Current I | Ampere, accepted range 0 to 100 |
| Timestamp / channel | Monotone per channel, exactly one reading per tick |
| Quality policy | Only GOOD with finite, in-domain values may reach paceval |

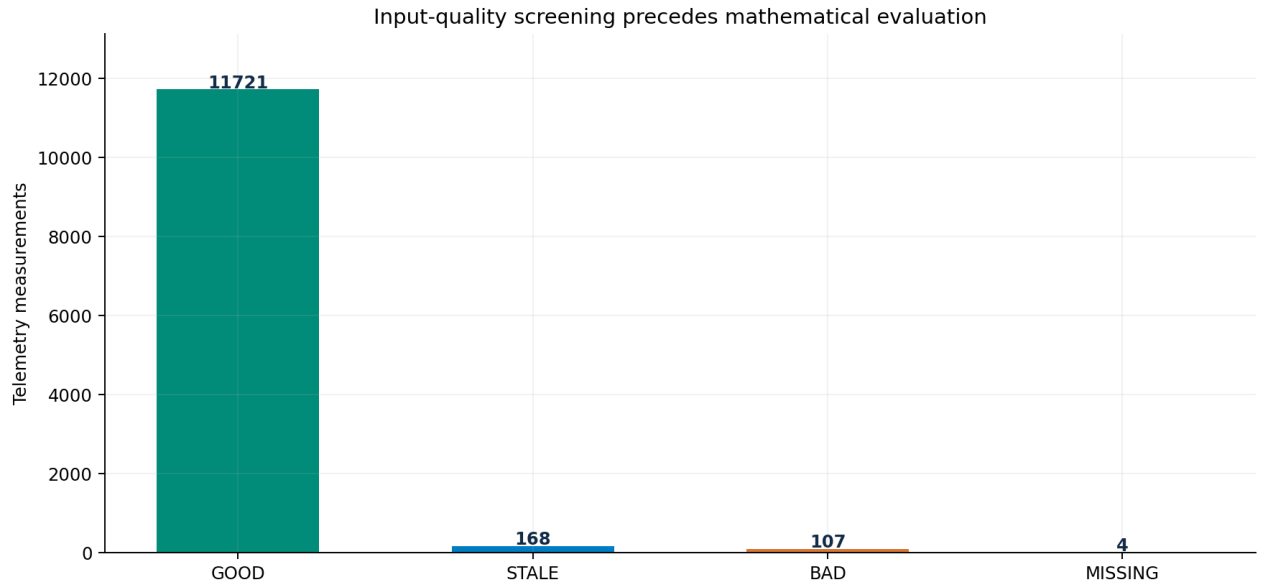



Figure 2. Actual quality-state counts from the synthetic CSV replay. Ineligible values are recorded as NO_DATA instead of silently interpreted as safe.

Quality is checked before batch submission. The demonstrator intentionally rejects a replay with duplicate/non-monotonic timestamps; this failure is included as a dedicated automated regression test.

# 3. Four explicit mathematical model families

For every rule, input normalization yields dimensionless terms t = (T-70)/40, v = V/2, and i = max(0, I-L)/8, where L is the rule-specific current limit. Coefficients are fixed per approved rule and published in `examples/rules.csv`.

| Family | Mathematical expression |
|---|---|
| Weighted linear | S = w1*t + w2*v + w3*i |
| Bounded nonlinear | S = tanh(w1*t + w2*v + w3*i) |
| Quadratic severity | S = w1*t² + w2*v² + w3*i² |
| Interaction model | S = w1*t + w2*v + w3*i + w4*t*v |

The real-valued identity max(0,z) = (z + abs(z))/2 is used for the positive-part term in this specific tested runtime. This is an algebraic equivalence, not an intrinsic limit on expression length or variables. Numeric differences near cancellation boundaries remain a separate validation matter.

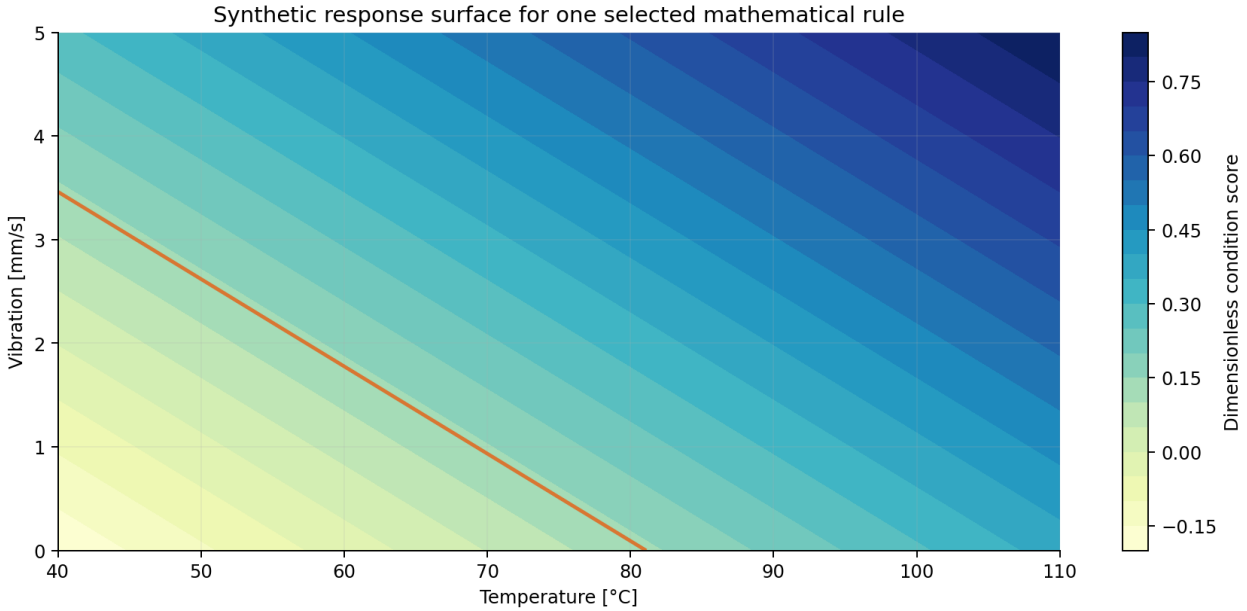



Figure 3. Synthetic score surface for one published rule at a fixed illustrative current. The contour represents its score threshold.

# 4. What paceval actually implements

The program includes the original `paceval_main.h` and compiles `paceval_main.cpp` together with `gateway_rules.cpp`. CMake links the separately supplied original Linux `.so`. The runtime library is not redistributed. The published SDK copyright and AGPL notices remain unchanged.

The tested direct API path initializes the engine, creates 100 distinct prepared computation objects, reuses each handle over validated batches, retrieves numerical results and per-item error status, and deletes the objects at shutdown. A separate interval-enabled graph is used solely for a TINC diagnostic.

| paceval API | Role in the demonstrator |
|---|---|
| InitializeLibrary / FreeLibrary | Engine lifecycle |
| CreateComputation | Build and approve each rule-specific execution graph |
| dGetComputationResultExt | Repeated batched evaluation with per-item status |
| dGetComputationResult | 24 pointwise TINC diagnostic calculations |
| DeleteComputation | Release old graph following a successful version update |

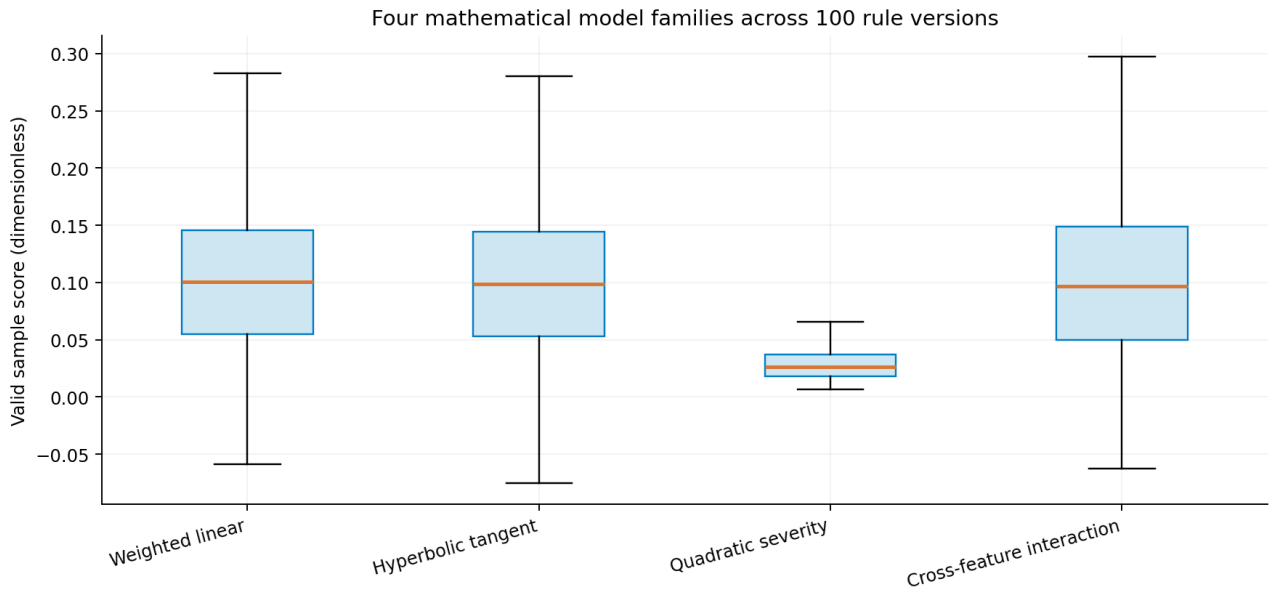



Figure 4. Actual evaluated score distributions across the four model families. These are mathematical scores, not calibrated failure probabilities.

# 5. Versioned rules and host-owned monitoring

A rule change is staged by creating a new paceval computation graph before swapping its handle and releasing the old graph. This single-threaded demonstration updates rule 17 from version 1.0 to 1.1 and leaves the other 99 rule objects unaffected. It does not claim multi-tenant atomic transaction semantics or tested concurrency.

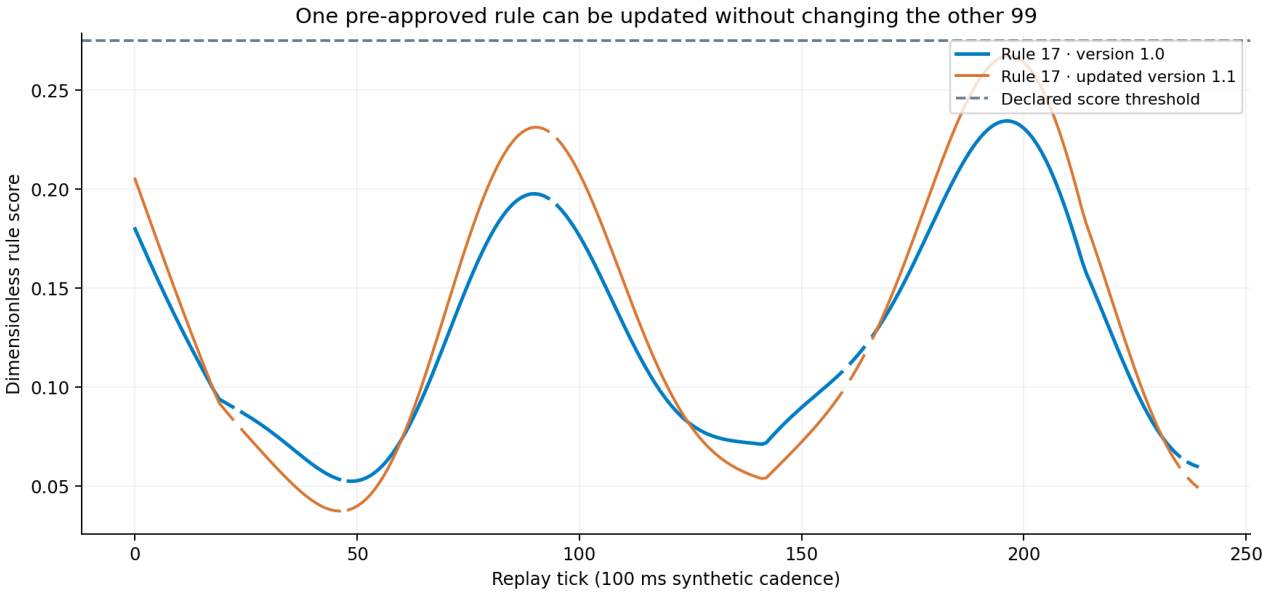



Figure 5. Score trajectories before and after the intentionally changed coefficients of one rule, with its unchanged threshold.

Monitoring decisions are not computed by the mathematical engine. In the C++ host, three consecutive valid readings above the score threshold activate an illustrative alert. An active alert clears when the score is below threshold minus 0.025. Stale/bad/missing inputs are reported as NO_DATA; they reset the example state rather than falsely asserting that a machine is safe.

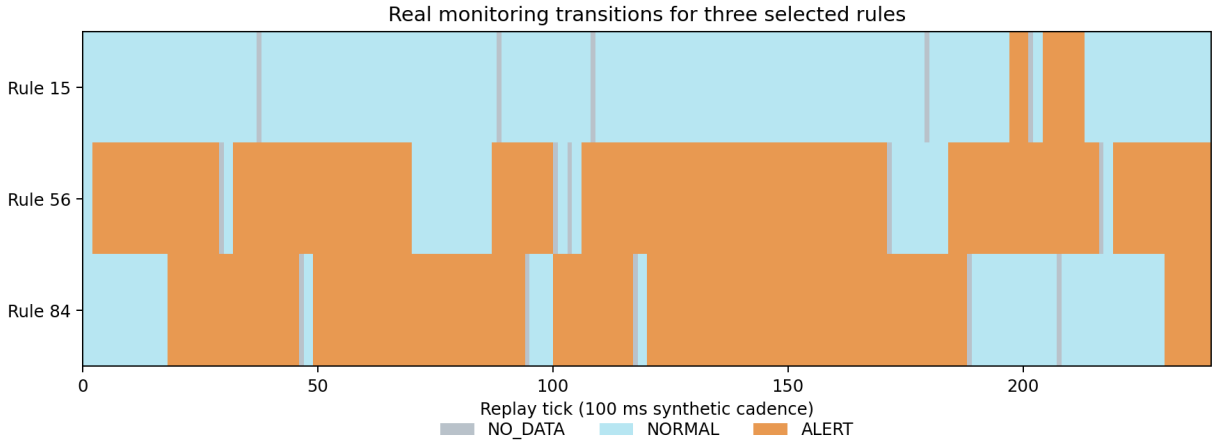



Figure 6. Illustrative alert states generated by the host-side persistence and hysteresis policy.

# 6. Numerical and functional findings

| Verified observation | Result |
|---|---|
| Raw replay | 12,000 measurements, 50 channels, 240 ticks |
| Individual rule graphs | 100 prepared, reused within the same process |
| Accepted / excluded evaluations | 23,442 accepted; 558 NO_DATA |
| Quality-related exclusions | 336 STALE, 214 BAD, 8 MISSING |
| Monitoring states | 22,586 NORMAL and 856 ALERT |
| Native C++ agreement | 23,442 within predeclared tolerance |
| High-precision independent audit | 600 checked at 90 decimal digits |
| Largest sampled absolute numerical error | Approx. 7.08 × 10^-17 |
| Updated graph isolation | Only the 240 records for rule 17 changed |

The audit compared every valid engine result to a separately coded C++ function. A second Python implementation uses `mpmath` and the published decimal inputs and coefficients. The tolerance is 2 × 10^-11 × (1 + |reference|). These are mathematical agreement tests, not measurements of alarm detection accuracy against fault labels.

A replay using batch sizes 1, 37 and 256 produced byte-identical CSV results. The controlled update likewise left all non-targeted rule rows unchanged. Tests passed with GCC, Clang, and with AddressSanitizer and UndefinedBehaviorSanitizer enabled.

# 7. TINC and independent interval audit

A dedicated computation graph was created with TINC enabled and evaluated at 24 valid samples of rule 0. The returned point value and interval endpoints were recorded and compared against a separate 90-digit reference using the published numeric inputs. For this specific test set, every interval contained its reference value. This is a sample-level observation, not a general proof of outward-rounding guarantees.

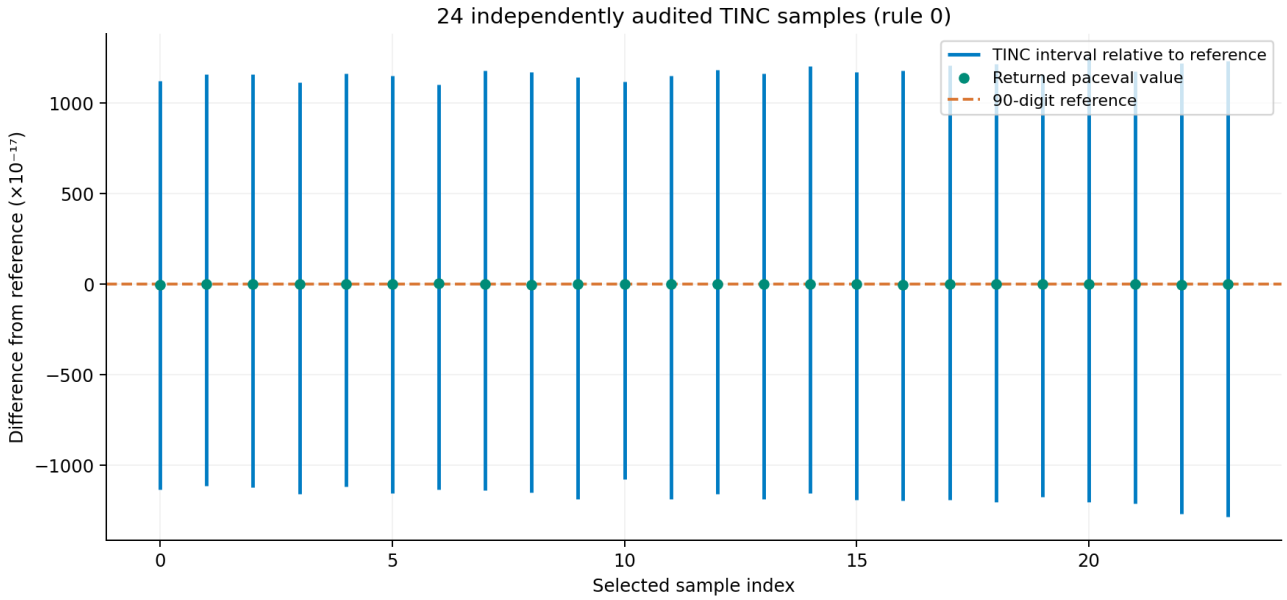



Figure 7. Difference to the high-precision reference. Vertical lines indicate the reported interval; the dashed line is the independently calculated reference.

The independent audit and all regression inputs are included in `src/validate_outputs.py`, `examples/tinc_samples.csv` and `examples/validation.json`. This allows the containment observation to be rechecked and prevents promotion of an unverified numerical claim.

# 8. Reproducibility, industrial next steps and sources

The release is a deliberately narrow C++ feasibility artefact. It demonstrates reusable programmable mathematics and controlled quality/decision ownership. It does not claim OPC UA/MQTT interoperability, real sensor validity, predictive power, guaranteed p99 latency, CPU-budget compliance, certified fail-safe behaviour, automatic actuation or measured FPGA/ASIC gains.

## Build and rerun (Linux)

```
export PACEVAL_LIB=/absolute/path/libpaceval_linux_sharedLIB.so

cmake -S src -B build -DCMAKE_BUILD_TYPE=Release

cmake --build build -j && ctest --test-dir build --output-on-failure

./build/gateway_rules --replay examples/telemetry.csv --out /tmp/replay.csv --batch 37

python3 src/validate_outputs.py --examples examples --output examples/replay_results.csv --tinc
examples/tinc_samples.csv
```

## Proposed follow-up experiment

A controlled gateway pilot should replace the CSV adapter with a supervised OPC UA or Sparkplug consumer, verify timestamp and quality semantics under reconnect/reorder, preserve historical rule versions and add signed model deployment. Before any safety-sensitive use, independently demonstrate system-level timing, resource limits, privilege separation, failure response, observability and plant-specific model validity. Hardware execution remains a later engineering question.

## Selected technical references

[1] paceval C/C++ API and computation lifecycle — https://paceval.com/api/

[2] OPC Foundation, UA Part 14: PubSub, released 31 October 2025 — https://profiles.opcfoundation.org/document/15

[3] Eclipse Foundation, Sparkplug specification and edge-node lifecycle — https://github.com/eclipse-sparkplug/sparkplug

[4] Eclipse Kura developer documentation: managed gateways and field-device drivers — https://eclipse-kura.github.io/kura/docs-develop/

[5] NIST SP 800-82 Revision 3: Guide to OT Security — https://csrc.nist.gov/pubs/sp/800/82/r3/final

Release scope: synthetic offline input; all implementation files and figures are included. The proprietary Linux runtime is user-supplied and the existing SDK source notices are preserved.